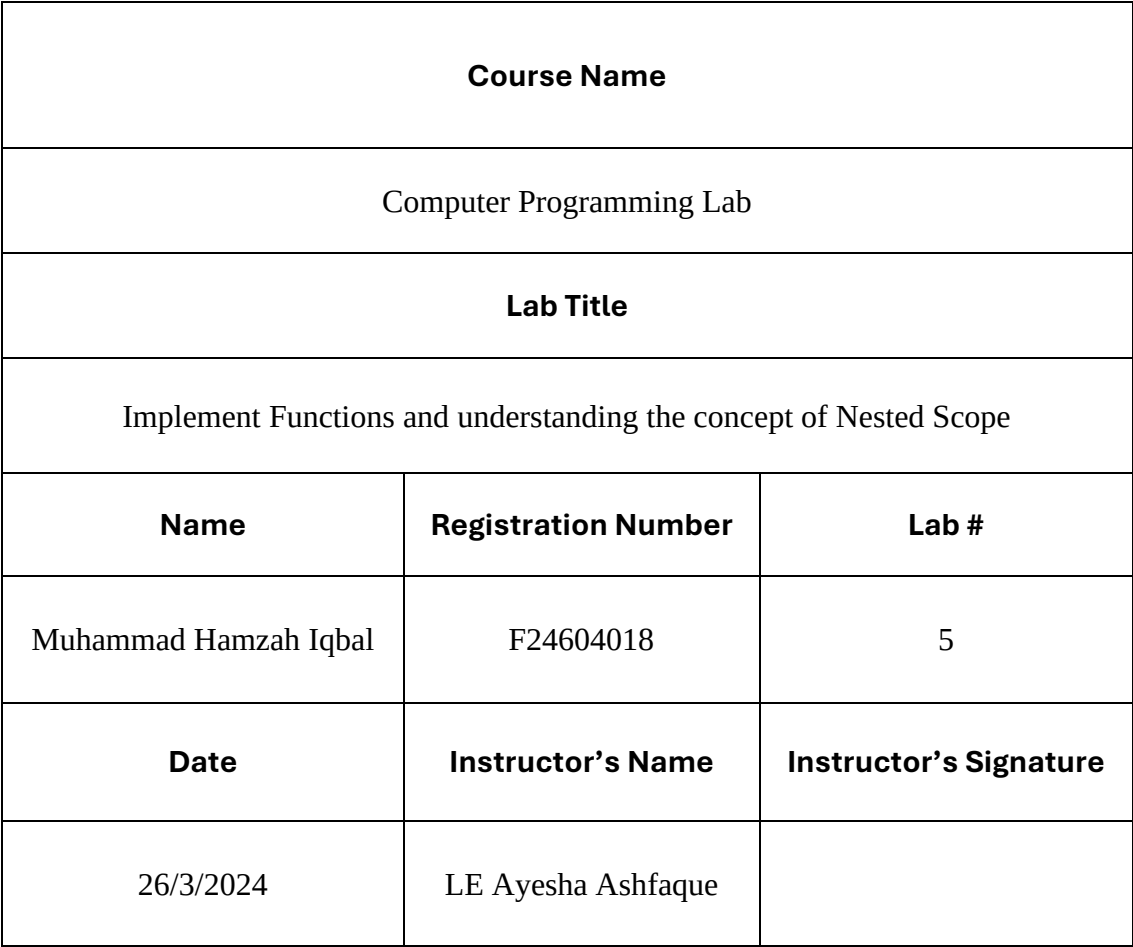




Objective:

- To have basic understanding of Functions in Python.
- To understand arguments and parameters

Theoretical Aspect

- **Definition & Purpose:** A function is a reusable block of code that performs a specific task, improving code organization and reusability. Defined using the def keyword.
- **Arguments & Parameters:** Functions accept inputs (positional, keyword, default, or arbitrary arguments) and can return values using the return statement.
- **Scope & Nesting:** Variables in functions follow local or global scope. Functions can be nested within other functions.
- **Built-in vs. User-defined:** Python provides built-in functions (abs(), round()), while users can define custom functions for specific tasks

Experimental Procedure

Task 01:

Complete the missing part in the code given below.

```
def my_function():  
    print("Hello From My Function!")
```

Solution:

Function was not called so after the program runs it displays nothing. To complete the missing part we just need to call the function below.

```
def my_function():  
    print("Hello From My Function!")  
  
my_function()
```

```

1 #-*- coding: utf-8 -*-
2 """
3
4 @author: M.Hamzah Iqbal
5 """
6 def my_function():
7     print("Hello from my Function!")
8
9 my_function()
10

```

Figure 1 Input

```

In [37]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/
untitled10.py', wdir='C:/Users/M.Hamzah Iqbal/One
Hello from my Function!

```

Figure 2 Output

Task 02

Complete the missing part in below code:

```

def sum_two_numbers(a, b):
    return a + b

```

Solution:

The function was not called, and the arguments a and b are not defined. The corrected code is shown below:

```

def sum_two_numbers(a,b):
    return a+b

print(sum_two_numbers(1,2))

```

```

3
4 @author: M.Hamzah Iqbal
5 """
6 def sum_two_numbers(a,b):
7     return a + b
8
9 print(sum_two_numbers(1,2))
10

```

Figure 3 Input

```

In [40]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/
untitled10.py', wdir='C:/Users/M.Hamzah Iqbal/One
3

```

Figure 4 Output

Task 03

Make your own functions.

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

Solution:

In this task I created my own function which it outputs the remainder of a number.

```
1  """  
2  @author: M.Hamzah Iqbal  
3  """  
4  def my_function_remainder(x):  
5      return x % 3  
6  
7  print(my_function_remainder(10))  
8  print(my_function_remainder(20))  
9  print(my_function_remainder(30))
```

Figure 5 Input

```
In [44]: runfile('C:/Users/M.Hamzah Iqbal/untitled10.py', wdir='C:/Users/M.Hamzah Iqbal')  
1  
2  
0
```

Figure 6 Output

Task 04

By running the given function of sum below,

- . Define functions for square, cube of given no. and make a function call
- a. Define function for square and cube.
- b. Input a value from user.
- c. Find its square and cube using function.

Function definition is here

```
def sum(arg1,arg2):
```

```
total=arg1+arg2 # Add both parameters and return them
```

```
print ("inside the function: ", total)
```

```
return total;
```

```
total = sum(10,20)    # calling sum function
```

```
print ("outside the function : ", total)
```

Solution:

```
1  # -*- coding: utf-8 -*-
2  """
3  @author: M.Hamzah Iqbal
4  """
5
6  num = int(input("Enter a number: "))
7
8  def square(num):
9      return num ** 2
10
11 def cube(num):
12     return num ** 3
13
14 print("Square of", num, "is:", square(num))
15 print("Cube of", num, "is:", cube(num))
16
17 def sum(arg1, arg2):
18     total = arg1 + arg2
19     print("Inside the function:", total)
20     return total
21 total = sum(10, 20)
22 print("Outside the function:", total)
```

Figure 7 Input

```
In [46]: runfile('C:/Users/M.Hamzah Iqbal/
untitled11.py', wdir='C:/Users/M.Hamzah Iq
Enter a number: 6
Square of 6 is: 36
Cube of 6 is: 216
Inside the function: 30
Outside the function: 30
```

Figure 8 Output

Task 05

Create the function that converts the PKR into US \$. It takes positional argument PKR and returns the converted amount.

Solution:

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Wed Mar 26 22:06:12 2025
4
5 @author: M.Hamzah Iqbal
6 """
7
8 def convert_pkr_to_usd(pkr):
9     exchange_rate = 277
10    usd = pkr / exchange_rate
11    return round(usd, 2)
12
13
14 pkr_amount = float(input("Enter amount in PKR: "))
15
16
17 usd_amount = convert_pkr_to_usd(pkr_amount)
18 print(f"{pkr_amount} PKR is approximately {usd_amount} USD.")
19
```

Figure 9 Input

```
In [47]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutech_2_Sem/Computer programming Lab/untitled12.py', wdi
OneDrive/Desktop/Nutech_2_Sem/Computer progr
Enter amount in PKR: 20000
20000.0 PKR is approximately 72.2 USD.
```

Figure 10 Output

Task 06

Create the following functions. (Separated Functions)

1. Sum, SUB, Mul & Div two integers with a value as a keyword argument. (they all are separated functions)
2. To calculate the percentage with 2 keyword argument & one Default argument with the value of 100.
3. To calculate the equation of line. Give slop as a positional argument. X should be Keyword argument and intercept should be default argument.
4. Create the function that return the whole square of a & b which are used as key word argument.
5. Create a function that takes two following list one as positional and other is keyword.

Hint:

Name:[]

Marks:[]

The output should look like while calling the functions is.

Raooof has 70 marks.

Nadia has 180 marks.

Mardan has 23 marks.

Naila has 90 marks.

Solution:

1.Sum, SUB, Mul & Div two integers with a value as a keyword argument. (they all are separate functions)

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6 def add(a, b):
7     return a + b
8
9 def sub(a, b):
10    return a - b
11
12 def mul(a, b):
13    return a * b
14
15 def div(a, b):
16    if b == 0:
17        return "Invalid"
18    return a / b
19
20 print("Sum:", add(a=5, b=3)) #Using KEYWORD ARGUMENT
21 print("Subtraction:", sub(a=10, b=4))
22 print("Multiplication:", mul(a=6, b=7))
23 print("Division:", div(a=20, b=5))
```

Figure 11 Input

```
In [48]: runfile('C:/Users/M.Hamz
Computer programming Lab/untitled
OneDrive/Desktop/Nutech_2_Sem/Com
Sum: 8
Subtraction: 6
Multiplication: 42
Division: 4.0
```

Figure 12 Output

2.To calculate the percentage with 2 keyword argument & one Default argument with the value of 100.

```
2
3
4 @author: M.Hamzah Iqbal
5 """
6
7 def calculate_percentage(obtained_marks, total_marks=100):
8     return (obtained_marks / total_marks) * 100
9
10 print("Percentage:", calculate_percentage(obtained_marks=80, total_marks=200))
11 print("Percentage (default total 100):", calculate_percentage(obtained_marks=80))
12
```

Figure 13 Input

```
In [49]: runfile('C:/Users/M.Hamzah Iq
Computer programming Lab/untitled14.py
OneDrive/Desktop/Nutech_2_Sem/Computer
Percentage: 40.0
Percentage (default total 100): 80.0
```

Figure 14 Output

3. To calculate the equation of line. Give slop as a positional argument. X should be Keyword argument and intercept should be default argument.

```
3
4 @author: M.Hamzah Iqbal
5 """
6
7 def equation_of_line(m, x=0, c=0):
8     return m * x + c
9
10 print("Equation result:", equation_of_line(2, x=3, c=5))
11 print("Equation with default c:", equation_of_line(4, x=2))
12
13
```

Figure 15 Input

```
In [50]: runfile('C:/Users/M.Hamzah Iq
Computer programming Lab/untitled14.py
OneDrive/Desktop/Nutech_2_Sem/Computer
Equation result: 11
Equation with default c: 8
```

Figure 16 Output

4. Create the function that return the whole square of a & b which are used as key word argument.

```
4 @author: M.Hamzah Iqbal
5 """
6
7 def whole_square(a, b):
8     return (a + b) ** 2
9
10 print("Whole Square:", whole_square(a=3, b=2))
11
```

Figure 17 Input

```
In [51]: runfile('C:/Users/M.Ha
Computer programming Lab/until
OneDrive/Desktop/Nutech_2_Sem/C
Whole Square: 25
```

Figure 18 Output

5. Create a function that takes two following list one as positional and other is keyword.

Hint:

Name:[]

Marks:[]

The output should look like while calling the functions is.

Raooof has 70 marks.

Nadia has 180 marks.

Mardan has 23 marks.

Naila has 90 marks.

```
2 """
3
4 @author: M.Hamzah Iqbal
5 """
6
7 def display_students(names, marks=[]):
8     for i in range(len(names)):
9         print(f"{names[i]} has {marks[i]} marks.")
10
11 names_list = ["Raooof", "Nadia", "Mardan", "Naila"]
12 marks_list = [70, 180, 23, 90]
13
14 display_students(names_list, marks=marks_list)
```

Figure 19 Input

```
In [52]: runfile('C:/Users/
Computer programming Lab/un
OneDrive/Desktop/Nutech_2_S
Raooof has 70 marks.
Nadia has 180 marks.
Mardan has 23 marks.
Naila has 90 marks.
```

Figure 20 Output

Task 07

Create the function calculator that takes 3-4 inputs and call all the function of task #6 except part 5.

Call the calculator function and check whether it gives your desired output or not.

Solution:

```
1 # -*- coding: utf-8 -*-
2 """
3
4 @author: M.Hamzah Iqbal
5 """
6 def add(a, b):
7     return a + b
8
9 def sub(a, b):
10    return a - b
11
12 def mul(a, b):
13    return a * b
14
15 def div(a, b):
16    return "Cannot divide by zero" if b == 0 else a / b
17
18 def calculate_percentage(obtained_marks, total_marks=100):
19    return (obtained_marks / total_marks) * 100
20
21 def equation_of_line(m, x=0, c=0):
22    return m * x + c
23
24 def whole_square(a, b):
25    return (a + b) ** 2
26
27 def calculator(a, b, operation, extra=None):
28    if operation == "add":
29        return add(a, b)
30    elif operation == "sub":
31        return sub(a, b)
32    elif operation == "mul":
33        return mul(a, b)
34    elif operation == "div":
35        return div(a, b)
36    elif operation == "percentage":
37        return calculate_percentage(a, b)
38    elif operation == "equation":
39        return equation_of_line(a, x=b, c=extra)
40    elif operation == "whole_square":
41        return whole_square(a, b)
42    else:
43        return "Invalid operation"
44
45 print("Addition:", calculator(10, 5, "add"))
46 print("Subtraction:", calculator(15, 7, "sub"))
47 print("Multiplication:", calculator(3, 6, "mul"))
48 print("Division:", calculator(20, 4, "div"))
49 print("Percentage:", calculator(40, 80, "percentage"))
50 print("Equation of Line:", calculator(2, 3, "equation", extra=5))
51 print("Whole Square:", calculator(4, 2, "whole_square"))
52
```

Console 1/A X

In [55]: runfile('C:/Users/M.Hamzah/Desktop/Task 7 Calculator.py', wdir='C:/Users/M.Hamzah/Desktop')
Addition: 15
Subtraction: 8
Multiplication: 18
Division: 5.0
Percentage: 50.0
Equation of Line: 11
Whole Square: 36

In [56]:

Figure 21 Task 7 Calculator

Task 08

Write a program in python for the given Pseudo code.

PSEUDO CODE:

1. Start program
2. Define a function with a value as input argument. Then print square and cube of the function.
3. Define another function and print only b.
4. Define another function and print only c.
5. Call the function within a function.
6. Print results
7. Stop program

Solution:



```
1 """
2
3 @author: M.Hamzah Iqbal
4 """
5 def calculate_values(num):
6     square = num ** 2
7     cube = num ** 3
8     print(f"Square: {square}, Cube: {cube}")
9     return square, cube
10
11 def print_b(b):
12     print(f"Value of b (Square): {b}")
13
14 def print_c(c):
15     print(f"Value of c (Cube): {c}")
16
17 def main_function(value):
18     square, cube = calculate_values(value)
19     print_b(square)
20     print_c(cube)
21
22 main_function(3)
23
```

Console 1/A X

In [58]: runfile('C:/Use
wdir='C:/Users/M.Hamzah
Square: 9, Cube: 27
Value of b (Square): 9
Value of c (Cube): 27

In [59]:

Figure 22 Input and Output of Task 8

Tupple tasks:

1. Write a program to implement Tuples.

PSEUDO CODE:

- Start
- First define the functions for tuples.
- Use for statement to check the tuples.
- Print the tuples.
- Increment the numbers and words.
- Get the maximum and minimum values from the numbers.
- Return the function
- Use assignments
- Call the function.
- End

Solution:

```
2 """
3 @author: M.Hamzah Iqbal
4 """
5 def tuple_operations():
6     numbers = (5, 10, 15, 20, 25)
7     words = ("apple", "banana", "cherry")
8
9     print("Numbers Tuple:", numbers)
10    print("Words Tuple:", words)
11
12    for num in numbers:
13        print("Number:", num)
14
15    for word in words:
16        print("Word:", word)
17
18    incremented_numbers = tuple(x + 1 for x in numbers)
19    print("Incremented Numbers:", incremented_numbers)
20
21
22    max_value = max(numbers)
23    min_value = min(numbers)
24    print("Max Value:", max_value)
25    print("Min Value:", min_value)
26
27    return numbers, words, incremented_numbers, max_value, min_value
28
29 result = tuple_operations()
30
31 print("Output of the Function:", result)
32
```

```
Console 1/A X
In [61]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Python/Tuple/Tuple.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Python/Tuple')
Numbers Tuple: (5, 10, 15, 20, 25)
Words Tuple: ('apple', 'banana', 'cherry')
Number: 5
Number: 10
Number: 15
Number: 20
Number: 25
Word: apple
Word: banana
Word: cherry
Incremented Numbers: (6, 11, 16, 21, 26)
Max Value: 25
Min Value: 5
Output of the Function: ((5, 10, 15, 20, 25), ('apple', 'banana', 'cherry'), (6, 11, 16, 21, 26), 25, 5)
In [62]:
```

Figure 23 Task 1 Tuple

2. Use the program of task 1 to use range and increment.

PSEUDO CODE:

- **Start**
- **Define list**
- **Use for statement from an initial value to the range of length of list.**
- **For loop will increment the variable from initial value to the end range of loop in each iteration.**
- **Print the total value.**
- **End**

Solution

```
1  """
2  """
3  @author: M.Hamzah Iqbal
4  """
5  def tuple_operations():
6      numbers = (5, 10, 15, 20, 25)
7      words = ("apple", "banana", "cherry")
8
9      print("Tuple of Numbers:", numbers)
10     print("Tuple of Words:", words)
11
12     numbers_incremented = tuple(num + 1 for num in numbers)
13     words_incremented = tuple(word + "s" for word in words)
14
15     print("Incremented Numbers:", numbers_incremented)
16     print("Incremented Words:", words_incremented)
17
18     print("Max Value:", max(numbers))
19     print("Min Value:", min(numbers))
20
21     return numbers_incremented
22
23
24 def range_increment_using_tuple():
25     numbers = tuple_operations()
26     total = 0
27
28     for i in range(len(numbers)):
29         total += numbers[i]
30
31     print("Total Sum using Tuple:", total)
32
33
34 range_increment_using_tuple()
35
```

Figure 24 Input

```
In [63]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/D
wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutec
Tuple of Numbers: (5, 10, 15, 20, 25)
Tuple of Words: ('apple', 'banana', 'cherry')
Incremented Numbers: (6, 11, 16, 21, 26)
Incremented Words: ('apples', 'bananas', 'cherrys')
Max Value: 25
Min Value: 5
Total Sum using Tuple: 80
```

Figure 25 Output

3. Write a program to check two tuple.

PSEUDO CODE:

- Start
- Write two lists.
- Compare the lists to check if they are equal.
- You can also check if a specific value is present in the list.
- End

Solution

```
1  """
2  """
3  @author: M.Hamzah Iqbal
4  """
5  def compare_tuples():
6      tuple1 = (1, 2, 3, 4, 5)
7      tuple2 = (1, 2, 3, 4, 5)
8
9      if tuple1 == tuple2:
10         print("The tuples are equal.")
11     else:
12         print("The tuples are not equal.")
13
14     value = int(input("Enter a value to check in the tuple: "))
15
16     if value in tuple1:
17         print(f"{value} is present in the tuple.")
18     else:
19         print(f"{value} is not in the tuple.")
20
21 compare_tuples()
```

Figure 26 Input

```
In [64]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/nutech_
wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/nutech_
The tuples are equal.
Enter a value to check in the tuple: 12
12 is not in the tuple.
```

Figure 27 Output

What will be the output of the following code snippet?

i.

```
init_tuple = ()
print (init_tuple. len  ())
```

```
File c:\users\m.hamzah iqbal\onedrive\desktop\nutech_
print (init_tuple.len())
AttributeError: 'tuple' object has no attribute 'len'
```

ii.

```
init_tuple_a = 'a',  
'b' init_tuple_b =  
( 'a', 'b')
```

```
print (init_tuple_a == init_tuple_b)
```

```
In [71]: runfile('C:/Users/  
wdir='C:/Users/M.Hamzah Iqbal'  
False
```

iii.

```
init_tuple_a = '1', '2'  
init_tuple_b = ('3', '4')
```

```
print (init_tuple_a + init_tuple_b)
```

```
In [72]: runfile('C:/Users/  
wdir='C:/Users/M.Hamzah Iqbal'  
( '1', '2', '3', '4')
```

iv.

```
init_tuple_a = 1, 2  
init_tuple_b = (3, 4)  
[print(sum(x)) for x in [init_tuple_a + init_tuple_b]]
```

```
In [73]: runfile('C:/Users/  
wdir='C:/Users/M.Hamzah Iqbal'  
10
```

v.

```
init_tuple = [(0, 1), (1, 2), (2, 3)]  
result = sum(n for _, n in init_tuple)
```

```
print(result)
```

```
In [74]: runfile('C:/Users/  
wdir='C:/Users/M.Hamzah Iqbal'  
6
```

vi.

```
l = [1, 2, 3]  
init_tuple = ('Python',) * (l.__len__() - l[::-1][0])  
print(init_tuple)
```

```
File c:\users\m.hamzah iqbal\onedrive\desktop\nutech_2_se  
init_tuple = ('Python',) * (1. len () - 1[::-1][0])  
AttributeError: 'list' object has no attribute 'len'
```

Discussion of Results

In this lab, we achieved the following:

- Implemented tuple operations, including indexing, slicing, and comparison.
- Defined functions for arithmetic operations (addition, subtraction, multiplication, division) using keyword arguments.
- Created a function to calculate square and cube of a number and successfully called functions within a function.
- Designed a PKR to USD converter using a positional argument and returned the converted amount.
- Implemented a calculator function that calls multiple arithmetic and mathematical operations.
- Used range and loops to iterate through tuples and lists, demonstrating increment operations.
- Successfully checked equality of two tuples and verified if a specific value is present.
- Explored list and tuple assignments, understanding immutability and mutability differences.
- Encountered assignment errors in slicing, reinforcing Python's tuple constraints.

Conclusions

- Python tuples and lists are essential data structures, offering different levels of flexibility and immutability.
- Tuples provide fixed, ordered storage, while lists allow modifications, making them suitable for various programming needs.
- Function calls and argument handling improve code reusability and efficiency in calculations and data processing.
- Using loops and range functions enhances iteration techniques, making operations on lists and tuples more effective